AlgoTrade

An AI-Powered Stock-Research Dashboard for Retail Investors
Design & Specification Document

Track	Track 3 — Software Product: AI Application for Investors
Product	Interactive web application (Streamlit) with an AI research engine
Domain	US public equities — fundamental analysis & comparison
Interface	Hebrew UI for non-expert investors
AI provider	Anthropic Claude (multi-agent) with server-side web search
Live demo	https://algotrade-yehoidoidan.streamlit.app
Submitted by	Yehonatan · Ido · Idan

1. Problem Definition

Retail investors who want to evaluate and compare US-listed companies face three persistent obstacles. **First, the data is fragmented:** valuation multiples, profitability ratios, balance-sheet health, analyst targets and breaking news each live in a different place and a different format, so building a single comparable picture of even three companies is slow, manual work. **Second, the data and tooling are English-first and assume domain expertise** — terms such as P/E, EV/EBITDA, ROE or current ratio are meaningless to most non-professional investors, and almost no quality tooling speaks Hebrew. **Third, raw numbers are not an answer:** knowing that one company trades at a P/E of 28 and another at 14 does not tell a beginner which is the better investment, for whom, or why.

AlgoTrade addresses this gap. It aggregates fundamental data for 10,000+ US tickers into one dashboard, explains every metric in Hebrew, and layers an AI research engine on top that reads the numbers, searches for current news, and produces a structured, plain-language comparison and recommendation tailored to the investor's profile (value, growth, or income). The goal is to lower the barrier to disciplined, fundamentals-based research for the Hebrew-speaking retail investor — turning scattered data into an explained decision.

Scope & intent. AlgoTrade is an educational research and comparison tool, not a brokerage and not personalised investment advice. It does not place trades or hold funds.

2. Solution Overview

The product is a single-page Streamlit application. The user selects one or more tickers from a searchable directory; the dashboard then presents the company across seven analytical views, while an always-available AI assistant in the sidebar answers free-language questions and helps the user discover and shortlist stocks. The headline capability is a one-click **AI comparison**: the user picks a basket of tickers and the system runs a multi-agent pipeline that analyses each company in parallel and synthesises a final ranked recommendation.

2.1 Dashboard views

- **Chart** — interactive price history (Plotly).
- **Financials** — income statement, balance sheet and cash-flow trends.
- **Valuation** — multiples (P/E, P/B, P/S, EV/EBITDA) over time vs. live TTM.
- **Compare** — side-by-side metric tables and historical charts for a basket of tickers, plus the AI comparison engine.
- **Analysts** — price targets, recommendation distribution and earnings estimates.
- **Holders** — institutional and insider ownership.
- **Profile** — company description, sector and key facts.

2.2 The AI value-add (mandatory feature)

AlgoTrade embeds two complementary AI capabilities:

- **Conversational research assistant** — a Hebrew chat in the sidebar. The user asks open questions ("find me defence-sector stocks", "how is this company doing?"); the assistant answers using Claude with live web search, and tags every US ticker it mentions (e.g. `[$AAPL]`) so the UI can render one-click buttons that add the stock to the comparison basket.
- **Multi-agent comparison engine** — given a basket of tickers, a dedicated agent analyses each company in parallel (fundamentals + current news), and a final synthesis agent produces a comparison table, a ranking, and recommendations broken down by investor profile.

2.3 Why an agentic design (and not RAG over filings)

The assignment suggests RAG over 10-K filings as one possible AI feature. AlgoTrade deliberately takes a different, complementary route: rather than retrieving passages from a single static filing, it combines **structured live fundamentals** (numeric ground truth) with **real-time web search** (current events the latest filing cannot contain), and orchestrates several specialised agents to turn both into a decision. This keeps answers current and grounded in numbers, and naturally scales to comparing many companies at once. RAG over EDGAR filings is identified as future work (Section 6) to add document-level grounding on top of this base.

3. System Architecture

AlgoTrade is a layered application: a Streamlit presentation layer, a data-access layer that aggregates and caches market data from three sources, and an AI orchestration layer built on the Anthropic Claude API. The diagram below shows the end-to-end flow of the AI comparison engine, from data collection through the parallel per-stock agents to the final synthesis.

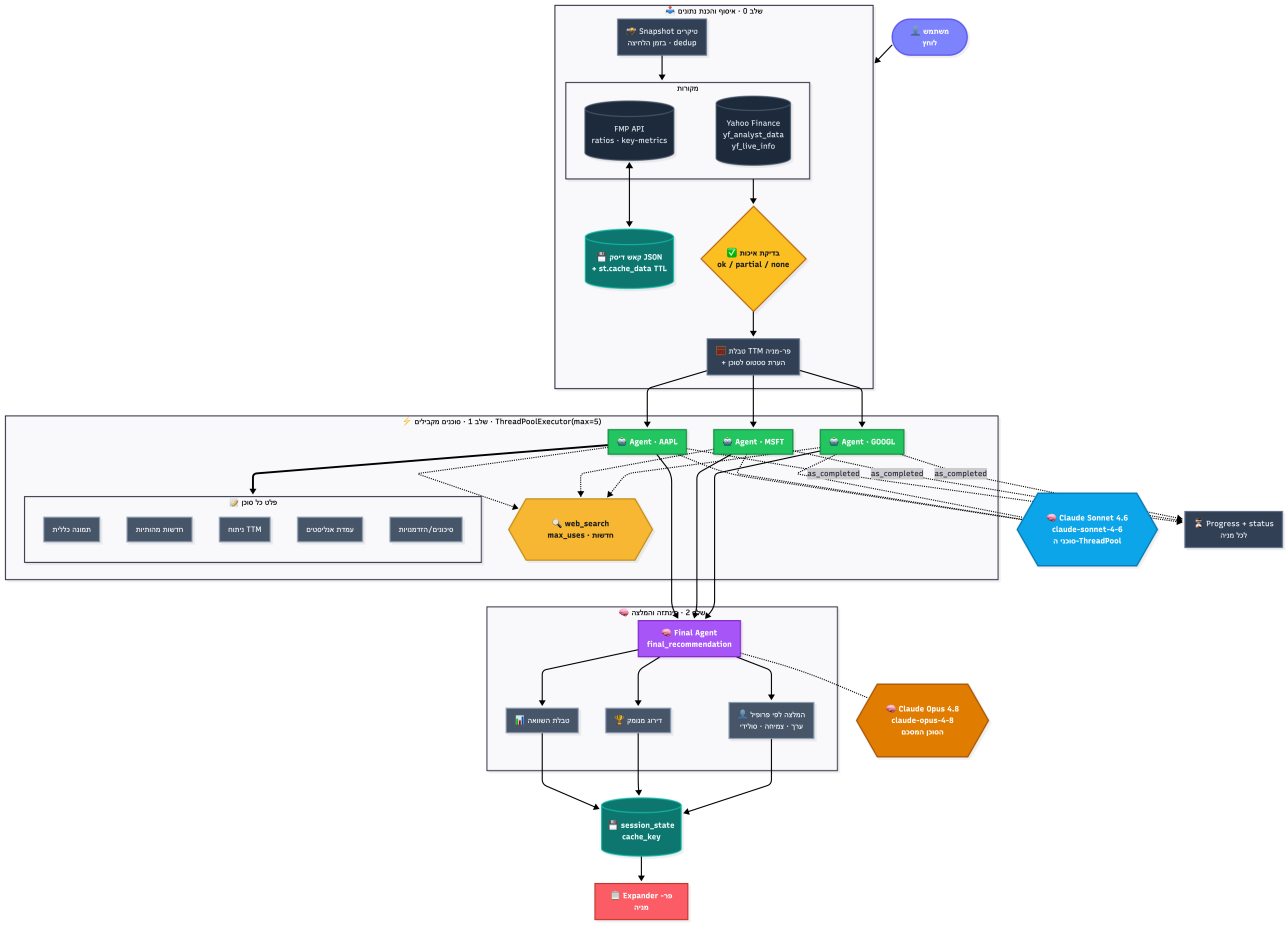

Figure 1 — AI comparison pipeline: Stage 0 data collection & quality gate · Stage 1 parallel per-stock agents (ThreadPoolExecutor) · Stage 2 final synthesis & recommendation. Per-stock agents run on Claude Sonnet 4.6; the final synthesis agent runs on Claude Opus 4.8.

3.1 Data layer

- **Ticker directory** — 10,000+ US tickers sourced from the SEC, cached daily; powers the searchable selector.
- **Financial Modeling Prep (FMP)** — historical `ratios` and `key-metrics` time series.
- **Yahoo Finance** (via `yfinance` / `curl_cffi`) — live TTM info and analyst data.
- **Caching** — two tiers: a JSON disk cache for API payloads plus Streamlit's `st.cache_data` TTL cache, to stay within free-tier rate limits and keep the UI responsive.

3.2 Data quality gate

Free data sources are incomplete and occasionally unavailable. Before any company reaches an agent, the system assesses each ticker's data and labels it **ok**, **partial**, or **none**. This status is embedded into the prompt the agent receives, so the model is explicitly told when fields are missing and can hedge its conclusions instead of fabricating numbers.

4. AI Engine & Models

4.1 Pipeline stages

- **Stage 0 — Collection & preparation.** Live TTM metrics and FMP fundamentals are fetched and assembled into a per-ticker TTM table, annotated with the data-quality status.
- **Stage 1 — Parallel per-stock agents.** One agent per ticker runs concurrently via a `ThreadPoolExecutor` (up to 5 workers). Each agent receives that company's TTM metrics and analyst data, calls the **web-search** tool for recent news, and returns a structured Hebrew summary: overview, material news, valuation read, analyst stance, and risks/opportunities. A progress bar reports each agent as it completes.
- **Stage 2 — Final synthesis.** A single synthesis agent receives all per-stock summaries and produces a comparison table, a ranking from most to least attractive, recommendations per investor profile (value / growth / income), and the principal risks.
- **Caching.** The full result is stored in `session_state` under a basket-specific key, so re-rendering does not re-invoke the model.

4.2 Models

Role	Model	Why
Sidebar assistant	Claude Sonnet 4.6	Fast, low-cost, conversational; pairs with web search.
Per-stock agents (Stage 1)	Claude Sonnet 4.6	Run in parallel and high-volume — speed and cost matter most.
Final synthesis (Stage 2)	Claude Opus 4.8	Single highest-stakes call — maximum reasoning for the recommendation.

Sonnet 4.6 (1M-token context; $3 / $15 per million input / output tokens) is used for the frequent, parallel calls where throughput and cost dominate. Opus 4.8 (1M-token context; $5 / $25 per million tokens) is reserved for the single final synthesis, where reasoning quality most affects the output. This split keeps quality high on the decision step while controlling cost on the fan-out.

4.3 Grounding & tools

- **Web search** — Anthropic's server-side web-search tool gives agents current prices and news beyond the model's training cutoff, reducing (though not eliminating) hallucination.
- **Structured numeric context** — every agent is given the actual TTM and historical metrics, so quantitative claims are anchored to real figures rather than recalled from memory.
- **Ticker tagging contract** — the system prompt requires every US ticker to be wrapped as `[$TICKER]`; a regex extracts these so the UI can render interactive controls. This is a lightweight, reliable structured-output convention.

4.4 Security

- API keys are read from `st.secrets` / environment variables — never hard-coded.
- `.env` and secrets files are git-ignored; only `.env.example` (placeholders) is committed.

5. Results

AlgoTrade is a working end-to-end product. A user can search 10,000+ US tickers, inspect any company across seven analytical views with every metric explained in Hebrew, chat with the AI assistant to discover stocks, and run a one-click multi-agent comparison that returns a ranked, profile-segmented recommendation grounded in live fundamentals and current news. The application is containerised (Docker / docker-compose) and runs as a single service.

A live demo is deployed on Streamlit Community Cloud: **https://algotrade-yehoidoidan.streamlit.app** (mockup deployment — the AI features require each visitor's own Anthropic API key, as the authors do not expose theirs, and market data falls back to a saved snapshot when the shared cloud IP is rate-limited; running locally shows full live data).

6. Risks & Caveats

Like any AI research tool, AlgoTrade operates within a few well-understood boundaries. These are handled deliberately in the design rather than left to chance:

- **A research aid, not advice.** AlgoTrade is built to support the investor's own judgement and speed up analysis — it is an educational tool, not personalised investment advice.
- **AI output is grounded, and worth a second look.** Every agent works from live numeric metrics and real-time web search, which keeps answers accurate and current; as with any AI-generated text, key conclusions are best confirmed against primary sources.
- **Robust to imperfect data.** The product relies on free market-data feeds, so a built-in quality gate flags any ticker with partial or missing data and passes that status to the AI, which hedges its conclusions accordingly instead of guessing.
- **Efficient by design.** A comparison fans out one model call per company; concurrency is capped and results are cached, keeping cost and latency predictable.

7. Future Work

- **RAG over SEC EDGAR filings** (10-K / 10-Q) for document-grounded Q&A on top of the current numeric + web-search base.
- **Portfolio tracking** and simple backtesting.
- **User accounts** for persisted baskets, chat history and preferences.
- **Cost controls** — prompt caching and cross-session caching of AI results.

Appendix — Technology Stack

Layer	Technology
UI	Streamlit, Plotly
Language / data	Python, pandas, numpy
Market data	SEC ticker directory, Financial Modeling Prep API, Yahoo Finance (yfinance, curl_cffi)
AI	Anthropic Claude (Sonnet 4.6 + Opus 4.8), server-side web search, ThreadPoolExecutor
Packaging	Docker, docker-compose